JAVA MINI PROJECT

INCLUSIVE LEARNING-MATERIAL SELECTION UNDER BUDGET

A 0/1 Knapsack approach using Dynamic Programming

Department of Computer Science and Engineering

Abstract

- Educational institutions often have limited budgets for learning materials.
- This project selects materials that provide the highest accessibility benefit without exceeding the available budget.
- The 0/1 Knapsack problem is solved using Dynamic Programming.
- Greedy comparison and Brute Force verification are also included.

Introduction

- Inclusive education aims to make learning resources accessible to every student.
- Learning materials have different costs and accessibility benefits.
- A limited budget makes it important to select the best combination.
- The problem is modelled as a 0/1 Knapsack optimization problem.

Objectives

- Accept material names, costs, benefits, and the total budget.
- Select the best combination within the available budget.
- Implement 0/1 Knapsack using Dynamic Programming.
- Compare the result with a Greedy benefit-to-cost method.
- Verify small test cases using Brute Force.

Algorithms Used

1. Dynamic Programming

Builds a table of best benefits for each item and budget capacity. Finds the optimal 0/1 selection.

2. Greedy Method

Sorts materials by benefit-to-cost ratio and selects items that fit. Used as a comparison baseline.

3. Brute Force

Checks possible combinations for small inputs to verify the optimal benefit returned by DP.

0/1 Knapsack – Dynamic Programming

For each material i and available budget w:

If cost[i] ≤ w:

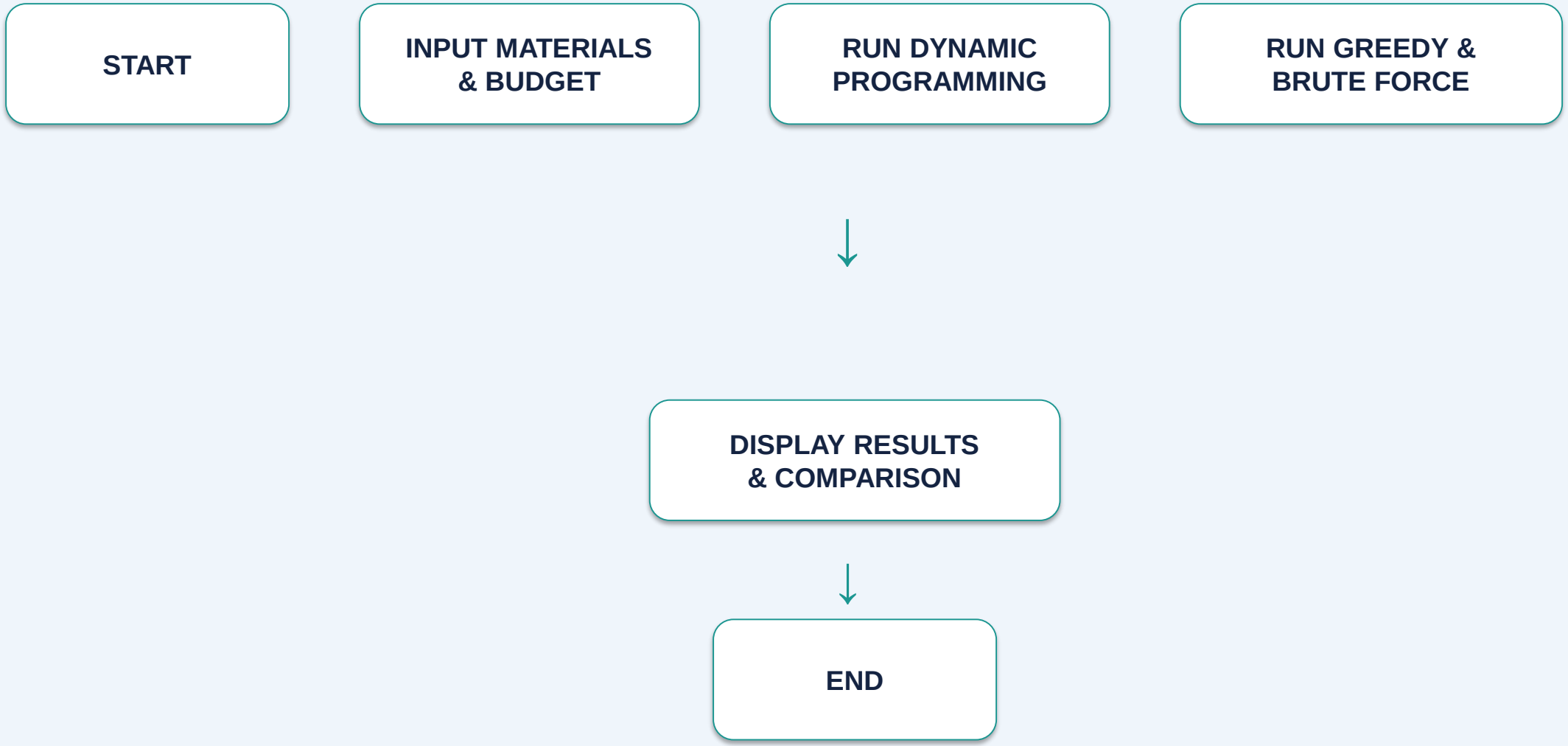
$$DP[i][w] = \max(benefit[i] + DP[i-1][w-cost[i]], DP[i-1][w])$$

Otherwise:

$$DP[i][w] = DP[i-1][w]$$

The final table entry gives the maximum accessibility benefit within the budget.

System Flowchart



Methodology

- Read the number of learning materials and collect each name, cost, and benefit.
- Read the total budget and validate the input values.
- Run Dynamic Programming to identify the optimal selection.
- Run Greedy and Brute Force algorithms for comparison.
- Display selected materials, total cost, benefit, and comparison.

Sample Results

Example input: Budget = 300

Learning Material	Cost	Benefit
Braille Learning Kit	200	90
Audio Learning Resources	150	80
Accessible Learning Software	100	85

Dynamic Programming: 175 | Greedy: 165 | Brute Force: 175

DP and Brute Force agree, confirming the optimal result for this small example.

Advantages

- Finds the optimal combination within the given budget.
- Considers both material cost and accessibility benefit.
- Compares Dynamic Programming with Greedy selection.
- Brute Force helps verify correctness for small inputs.
- Supports more informed allocation of learning resources.

Conclusion

- The project models learning-material selection as a 0/1 Knapsack problem.
- Dynamic Programming identifies the maximum benefit within the budget.
- Greedy is useful for comparison but may not always be optimal.
- Brute Force verification supports confidence in DP for small test cases.

Future Work

- Develop a graphical user interface.
- Store material details and selection history in a database.
- Add filters for different accessibility needs.
- Support larger datasets and detailed reporting.
- Explore multi-budget or multi-criteria resource allocation.

Complexity Analysis

Dynamic Programming

Time: O(n × W)
Space: O(n × W)
n = number of materials
W = budget capacity

Greedy

Time: O(n log n)
Main cost is sorting materials by benefit-to-cost ratio.

Brute Force

Time: $O(n \times 2^n)$
Suitable only for small inputs because it checks subsets.

THANK YOU

Questions & Discussion